PRN232 – ASSIGNMENT 02

Distributed and Intelligent News Management System

1. Introduction

This assignment requires students to develop a **Distributed News Management System (FUNewsManagementSystem v2)** that includes four independent components: **Core API**, **Analytics API**, **AI API**, and **Frontend**.
The system must demonstrate the ability to design a distributed architecture, communicate via **HttpClient**, implement **JWT authentication**, perform background processing using a **Background Worker**, apply **data caching**, integrate **AI-based Tag suggestions**, and provide an interactive dashboard using **Chart.js**.

2. Objectives

- Apply **.NET 8**, **Entity Framework Core**, **ASP.NET Core Web API**, and **Razor Pages / MVC**.
- Design a distributed system consisting of four projects: **Core API**, **Analytics API**, **AI API**, and **Frontend**.
- Integrate **HttpClientFactory** and **DelegatingHandler**.
- Build a **Background Worker (HostedService)** to periodically call APIs and store cached data.
- Create an **AI API** to automatically suggest tags (keywords) based on the article's content.
- Implement **Refresh Token Flow** to issue new JWTs when they expire.
- Add **Audit Logging** to record CRUD actions performed by users.
- Build a **SignalR Notification System** to display real-time notifications when new articles are created.
- Enable **Excel Report Export** from the dashboard.
- Design a **responsive and user-friendly interface** with clear error handling and alerts.

3. Project Requirements

3.1 Core API

- Implement CRUD operations for **Account**, **Category**, **Tag**, and **NewsArticle** using **Entity Framework Core**.
- Support **JWT Authentication** and issue **Refresh Tokens** via the endpoint `/api/auth/refresh`.

- Allow image uploads for articles, validating file type and size.
- Add **Audit Logging** to record user actions (User, Action, Entity, Before/After JSON).
- Enforce business rules: prevent deletion of related records.
- Return clear JSON responses and appropriate HTTP status codes.

3.2 Analytics API

- Main endpoints:
  - `/api/analytics/dashboard`: Display total articles by category and status.
  - `/api/analytics/trending`: List trending (most viewed) articles.
  - `/api/recommend/{id}`: Suggest related articles based on category or tags.
- Support advanced filtering by date, category, and author.
- Allow **Excel Report Export** via `GET /api/analytics/export`.

3.3 AI API

- Endpoint: `/api/ai/suggest-tags`
- Input: JSON `{ "content": "..." }`
- Output: Suggested tags (optionally with confidence scores).
- Can connect to the real **OpenAI API** or simulate keyword extraction manually.
- Maintain a **Learning Cache** to remember tags frequently selected by users.

3.4 Frontend (MVC / Razor Pages)

- Must not access the database directly; all communication is via **HttpClient**.
- Manage multiple **HttpClient** instances for Core, Analytics, and AI APIs.
- Implement **Polly Retry Policy** for API calls and show alerts when errors occur.
- Include a **Background Worker (HostedService)** to refresh cached data every 6 hours.
- Use **Chart.js** to render dashboards; perform CRUD operations using **Bootstrap Modals**.
- Include **Offline Mode** to read data from cache or local JSON when the API is down.
- Integrate **SignalR Notification System** to display live notifications of new articles.

4. SCREEN-LEVEL REQUIREMENTS

4.1 Login Page

- Function: Users enter email and password to log in.
- APIs: `POST /api/auth/login`, `POST /api/auth/refresh`
- Requirements:
  - Save `access_token` and `refresh_token` to session upon successful login.
  - Show Toast/Alert message when login fails.
  - Automatically refresh token before expiration.

4.2 Dashboard (Admin)

- Function: Display article statistics, categories, and authors.
- APIs: `GET /api/analytics/dashboard`, `GET /api/analytics/trending`
- Requirements:
  - Display **Chart.js** graphs (Pie, Bar).
  - Filter by date, category, and status.
  - Add "Export Excel" button (`GET /api/analytics/export`).

4.3 News List (Staff)

- Function: List all articles.
- API: `GET /api/news`
- Requirements:
  - Support pagination, search, and sort by creation date.
  - Columns: Title, Category, Created Date, Status.
  - Use color indicators for Active/Inactive posts.

4.4 Create / Edit News

- Function: Create or update an article.
- API: `POST /api/news`, `PUT /api/news/{id}`
- Requirements:
  - Use **Bootstrap Modals** for the form.
  - Validate Title, Content, Category, and Tag fields.
  - Call **AI Suggestion API** for automatic tag recommendations.

4.5 News Detail

- Function: Display article details and related articles.
- APIs: `GET /api/news/{id}`, `GET /api/recommend/{id}`
- Requirements:
  - Show up to 3 related articles based on category or tag.
  - Use a clean and responsive reading layout.

4.6 Category Management

- Function: Manage article categories.
- APIs: `GET /api/category`, `POST /api/category`, `PUT /api/category/{id}`, `DELETE /api/category/{id}`
- Requirements:
  - Prevent deletion if the category contains articles.
  - Toggle `IsActive` status.
  - Show the number of articles per category.

4.7 Tag Management

- Function: Manage tags.

- APIs: `GET /api/tag`, `POST /api/tag`, `PUT /api/tag/{id}`, `DELETE /api/tag/{id}`
- Requirements:
  - Prevent duplicate tag names.
  - Enable tag search.
  - Display all articles using each tag.

4.8 Account Management (Admin)

- Function: Manage system accounts.
- APIs: `GET /api/account`, `POST /api/account`, `PUT /api/account/{id}`, `DELETE /api/account/{id}`
- Requirements:
  - Do not delete accounts that created articles.
  - Require old password for password changes.
  - Filter accounts by role (Admin / Staff).

4.9 AI Tag Suggestion

- Function: Suggest tags based on content.
- API: `POST /api/ai/suggest-tags`
- Requirements:
  - Input content → click "Suggest Tag".
  - Display suggested tags as Chips or Badges.
  - Allow quick selection to attach tags to the article.

4.10 Notification Center

- Function: Show real-time notifications when new articles are published.
- API / SignalR Hub: `/hubs/notifications`
- Requirements:
  - Display a Toast or 🔔 icon notification when new articles are added.
  - Maintain at least 10 recent notifications.

4.11 Offline Mode

- Function: Operate when the system loses API connectivity.
- Data Source: Cache or local JSON.
- Requirements:
  - Detect when APIs are unreachable.
  - Show banner "Offline Mode".
  - Display cached data, but disable CRUD operations.

4.12 Audit Log (Admin)

- Function: Display audit history of data changes.
- API: `GET /api/auditlog`

- Requirements:
    - Show User, Action, Entity, and Timestamp.
    - Display Before/After data in JSON format.
    - Allow filtering by user or entity type.

General Notes

- Every CRUD operation must show a confirmation prompt.
- All API calls must display a **Loading Indicator**.
- The UI must be **responsive**, clear, and consistent in color and style.
- All data must be retrieved via **HttpClient**, not direct database access.

5. UX & Functional Requirements

- Display a **Loading Indicator** for all API operations.
- Show **Toast/Alert** for success or failure.
- Use a **Responsive Design** compatible with desktop and mobile.
- Automatically refresh **JWT** before expiration.
- Log all API requests (endpoint, time, status).
- Support **Accessibility**: keyboard navigation, high color contrast.
- **Performance:** API response time < 1 second; caching optimized for speed.

6. Submission Guidelines

- `FUNewsManagement_CoreAPI.sln`
- `FUNewsManagement_AnalyticsAPI.sln`
- `FUNewsManagement_AIAPI.sln`
- `FUNewsManagement_FE.sln`
- `FUNewsManagement.sql` (Database)
- `README.md` describing setup, API URLs, and test accounts.
- Screenshots: API responses, AI Tag Suggestion, Dashboard.

Note: Additional tables or fields may be added to the database if necessary.